1. True/False Questions

Problem 1.1 False (if for e.g. you flip images vertically you might change meaning like 6 becomes 9)

Problem 1.2 False (Dropout is a reg technique; it also increases #epochs for training to converage)

Problem 1.3 False (Doing both reg and dropout might lead to underfitting)

Problem 1.4 True (L1 reg region has sharp not-differentialbe corners at zero but L2 has round ones)

Problem 1.5 True (Helps avoid only non-linear layer passing. allows directly flow though network hence less chaotic landscape)

2. Computation Questions

Problem 2.1 ((100 * 100 * 3) * (100)) + (100) = 3,000,000 + 100 = 3,000,1000

Problem 2.2 (3 * 3 * 3 + 1) * 100 = 28 * 100 = 2,800

Problem 2.3 (1 * 1 * 16 + 1) = 17

Problem 2.4 out = ((100 - 5 +2(0))/(1)) + 1 = 96 96 * 96 * 32

Problem 2.5 Standard conv MACs: 3×3×M×N×D_F×D_F Depthwise separable MACs: 3×3×M×D_F×D_F + D_F×D_F×M×N Speedup = (3×3×M×N×D_F×D_F) / (3×3×M×D_F×D_F + D_F×D_F×M×N) = 1 / (1/N + 1/9) ≈ 9 (when N is large)

3 Lab (1): Training SimpleNN for CIFAR-10 classification

```
# define the SimpleNN mode;
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 8, 5)     # 8  * (3 * 5 * 5 + 1)     =
608
        self.conv2 = nn.Conv2d(8, 16, 3)    # 16 * (8 * 3 * 3 + 1)     =
1,168
        self.fc1   = nn.Linear(16*6*6, 120) # 120 * (16 * 6 * 6 + 1)   =
69,240
        self.fc2   = nn.Linear(120, 84)     # 84 * (120 + 1)           =
10,164
        self.fc3   = nn.Linear(84, 10)      # 10 * (84 + 1)            =
850

    def forward(self, x):
        out = F.relu(self.conv1(x))
        out = F.max_pool2d(out, 2)
        out = F.relu(self.conv2(out))
```

```
        out = F.max_pool2d(out, 2)
        out = out.view(out.size(0), -1)
        out = F.relu(self.fc1(out))
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

(a)

```
# sanity check for the correctness of SimpleNN

net = SimpleNN().to("cpu")
# dummpy CIFAR-10 batch: 4 RGB Images, 32 x 32
data = torch.randn(4, 3, 32, 32).to("cpu")
# forward pass "data" through "net" to get output "out"
out = net(data)
# check output shape
assert(out.detach().cpu().numpy().shape == (4, 10))

##############################################
conv_fc_params = 0
for layer in net.modules():
    if isinstance(layer, (nn.Conv2d, nn.Linear)):
        conv_fc_params += sum(p.numel() for p in layer.parameters())
# 608 + 1,168 + 69,240 + 10,164 + 850 = 82,030
assert(conv_fc_params == 82030)
```

I passed a dummy CIFAR-10 batch with shape (4, 3, 32, 32) through the network. Since CIFAR-10 has 10 classes, the output should have shape (4, 10).

I also checked the number of parameters in all convolutional and fully connected layers. Using the formulas

- Conv2d params = out_channels * (in_channels * kernel_h * kernel_w + 1)
- Linear params = out_features * (in_features + 1)

the expected total is: 608 + 1,168 + 69,240 + 10,164 + 850 = 82,030

Since the output shape is (4, 10) and the parameter count is 82,030, the implementation is consistent with the expected architecture.

(b)

```
# useful libraries
import torchvision
import torchvision.transforms as transforms

# ############################################
# # specify preprocessing function
```

```
transform_train = transforms.Compose([
    transforms.ToTensor(), # convert PIL images to tensor/image tensor
    transforms.Normalize(
        mean=[0.4914, 0.4822, 0.4465],
        std=[0.2023, 0.1994, 0.2010]
    )
])

transform_val = transforms.Compose([
    transforms.ToTensor(), # convert PIL images to tensor/image tensor
    transforms.Normalize(
        mean=[0.4914, 0.4822, 0.4465],
        std=[0.2023, 0.1994, 0.2010])
])
```

I used two preprocessing operations. First, ToTensor() converts the raw PIL image into a PyTorch tensor and scales pixel values from [0, 255] to [0, 1]. Second, Normalize() standardizes each RGB channel using the CIFAR-10 mean and standard deviation. This helps make the input distribution more stable, which usually improves neural network training.

(c)

```
################################################
# construct dataset
train_set = CIFAR10(
    root=DATA_ROOT,
    mode='train',
    download=True,
    transform=transform_train
)
val_set = CIFAR10(
    root=DATA_ROOT,
    mode='val',
    download=True,
    transform=transform_val
)

# construct dataloader
train_loader = DataLoader(
    train_set,
    batch_size=TRAIN_BATCH_SIZE,
    shuffle=True,
    num_workers=4
)
val_loader = DataLoader(
    val_set,
    batch_size=VAL_BATCH_SIZE,
    shuffle=True,
    num_workers=4
```

```
)
#############################################
```

(d)

```
#############################################
device = "cuda" if torch.cuda.is_available() else ("mps" if
torch.backends.mps.is_available() else "cpu")
net = SimpleNN().to(device)
print(next(net.parameters()).device) # cuda:0
#############################################
```

```
!nvidia-smi # ... 0  Tesla T4 ...
```

I checked GPU availability in PyTorch using torch.cuda.is_available(). Since CUDA was detected on the system, the device was set to "cuda". After initializing the model and transferring it using net = SimpleNN().to(device), I verified its deployment by printing next(net.parameters()).device, which returned cuda:0. I also run !nvidia-smi and confirmed the presence of an NVIDIA Tesla T4 GPU (Driver Version: 580.82.07, CUDA Version: 13.0). This verifies that the model parameters are successfully assigned to GPU device 0 for computation.

(e)

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=INITIAL_LR, momentum=MOMENTUM,
weight_decay=REG)
```

(f)

```
# some hyperparameters
# total number of training epochs
EPOCHS = 30

# the folder where the trained model is saved
CHECKPOINT_FOLDER = "./saved_model"

# start the training/validation process
# the process should take about 5 minutes on a GTX 1070-Ti
# if the code is written efficiently.
best_val_acc = 0
current_learning_rate = INITIAL_LR

print("==> Training starts!")
```

```
print("="*50)
for i in range(0, EPOCHS):
    ########################
    # your code here
    # switch to train mode
    net.train()

    ########################

    print("Epoch %d:" %i)
    # this help you compute the training accuracy
    total_examples = 0
    correct_examples = 0

    train_loss = 0 # track training loss if you want

    # Train the model for 1 epoch.
    for batch_idx, (inputs, targets) in enumerate(train_loader):
        ####################################
        # your code here
        # copy inputs to device
        inputs = inputs.to(device)
        targets = targets.to(device)

        # compute the output and loss
        outputs = net(inputs)
        loss = criterion(outputs, targets)

        # zero the gradient
        optimizer.zero_grad()

        # backpropagation
        loss.backward()

        # apply gradient and update the weights
        optimizer.step()

        # count the number of correctly predicted samples in the current
batch
        train_loss += loss.item()

        _, predicted = outputs.max(1)
        total_examples += targets.size(0)
        correct_examples += predicted.eq(targets).sum().item()

        ####################################

    avg_loss = train_loss / len(train_loader)
    avg_acc = correct_examples / total_examples
    print("Training loss: %.4f, Training accuracy: %.4f" %(avg_loss,
avg_acc))

    # Validate on the validation dataset
    ########################
```

```
    # your code here
    # switch to eval mode
    net.eval()

    ######################

    # this help you compute the validation accuracy
    total_examples = 0
    correct_examples = 0

    val_loss = 0 # again, track the validation loss if you want

    # disable gradient during validation, which can save GPU memory
    with torch.no_grad():
        for batch_idx, (inputs, targets) in enumerate(val_loader):
            ###################################
            # your code here
            # copy inputs to device
            inputs = inputs.to(device)
            targets = targets.to(device)

            # compute the output and loss
            outputs = net(inputs)
            loss = criterion(outputs, targets)

            # count the number of correctly predicted samples in the
current batch
            val_loss += loss.item()

            _, predicted = outputs.max(1)
            total_examples += targets.size(0)
            correct_examples += predicted.eq(targets).sum().item()
            ###################################

    avg_loss = val_loss / len(val_loader)
    avg_acc = correct_examples / total_examples
    print("Validation loss: %.4f, Validation accuracy: %.4f" % (avg_loss,
avg_acc))

    # save the model checkpoint
    if avg_acc > best_val_acc:
        best_val_acc = avg_acc
        if not os.path.exists(CHECKPOINT_FOLDER):
           os.makedirs(CHECKPOINT_FOLDER)
        print("Saving ...")
        state = {'state_dict': net.state_dict(),
                'epoch': i,
                'lr': current_learning_rate}
        torch.save(state, os.path.join(CHECKPOINT_FOLDER, 'simplenn.pth'))

    print('')

print("="*50)
```

```
print(f"==> Optimization finished! Best validation accuracy:
{best_val_acc:.4f}")
```

(g)

The initial loss before training is expected to be close to ln(10) ≈ 2.3026, because CIFAR-10 has 10 classes and an untrained model predicts roughly uniform probabilities.

During training, the training acc steadily increases from about 30% to 88.6%, and the training loss keeps decreasing. However, the val acc improves only up to about 66.3% around epoch 12 and then stops improving, while validation loss increases.

A problem with the current training pipeline is that it trains for all 30 epochs without learning rate decay, early stopping, or data augmentation, so the model begins to overfit after the validation acc peaks.

(h)

```
EPOCHS = 30
CHECKPOINT_FOLDER = "./saved_model"

DECAY_EPOCHS = 10 # parameter need for lr decay
DECAY = 0.5 # parameter need for lr decay

best_val_acc = 0
current_learning_rate = INITIAL_LR

for i in range(0, EPOCHS):
    if i % DECAY_EPOCHS == 0 and i != 0:
        current_learning_rate = current_learning_rate * DECAY
        for param_group in optimizer.param_groups:
            param_group['lr'] = current_learning_rate
        print("Current learning rate has decayed to %f"
%current_learning_rate)
# rest of code ...
```

- 67.12% (achieved at epoch 10 right after the first decay step to 0.005).
- Whenever the lr dropped (epochs 10 and 20), the training loss decreased immediately, showing the model was learning better with smaller step sizes.
- Even though the model started over-memorizing the training data later on, dropping the lr early allowed it to settle into better weights before that happened.

4 Lab (2): Improving the training pipeline

Helper function

```
def train_model(net, train_loader, val_loader, device,
                 epochs=30, lr=0.01, momentum=0.9, reg=1e-4,
                 checkpoint_name="model",
checkpoint_folder="./saved_model"):

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum,
weight_decay=reg)

    best_val_acc = 0
    history = {"train_loss": [], "train_acc": [], "val_loss": [],
"val_acc": []}

    print(f"==> Training starts! ({checkpoint_name})")
    print("=" * 50)
    for i in range(epochs):
        net.train()
        print("Epoch %d:" % i)
        total_examples, correct_examples, train_loss = 0, 0, 0

        for inputs, targets in train_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = net(inputs)
            loss = criterion(outputs, targets)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            _, predicted = outputs.max(1)
            total_examples += targets.size(0)
            correct_examples += predicted.eq(targets).sum().item()

        avg_loss = train_loss / len(train_loader)
        avg_acc = correct_examples / total_examples
        history["train_loss"].append(avg_loss)
        history["train_acc"].append(avg_acc)
        print("Training loss: %.4f, Training accuracy: %.4f" % (avg_loss,
avg_acc))

        net.eval()
        total_examples, correct_examples, val_loss = 0, 0, 0
        with torch.no_grad():
            for inputs, targets in val_loader:
                inputs, targets = inputs.to(device), targets.to(device)
                outputs = net(inputs)
                loss = criterion(outputs, targets)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                total_examples += targets.size(0)
```

```
                correct_examples += predicted.eq(targets).sum().item()

        avg_loss = val_loss / len(val_loader)
        avg_acc = correct_examples / total_examples
        history["val_loss"].append(avg_loss)
        history["val_acc"].append(avg_acc)
        print("Validation loss: %.4f, Validation accuracy: %.4f" %
(avg_loss, avg_acc))

        if avg_acc > best_val_acc:
            best_val_acc = avg_acc
            if not os.path.exists(checkpoint_folder):
                os.makedirs(checkpoint_folder)
            torch.save({'state_dict': net.state_dict(), 'epoch': i, 'lr':
lr},
                       os.path.join(checkpoint_folder, f"
{checkpoint_name}.pth"))
        print('')

    print("=" * 50)
    print(f"==> Optimization finished! Best validation accuracy:
{best_val_acc:.4f}")
    return history, best_val_acc
```

(a)

```
# Augmented transform
transform_train_aug = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(), # convert PIL images to tensor/image tensor
    transforms.Normalize(
        mean=[0.4914, 0.4822, 0.4465],
        std=[0.2023, 0.1994, 0.2010]
    )
])
```

```
# construct dataset
train_set_aug = CIFAR10(
    root=DATA_ROOT,
    mode='train',
    download=True,
    transform=transform_train_aug    # your code
)

# construct dataloader
train_loader_aug = DataLoader(
    train_set_aug,
    batch_size=TRAIN_BATCH_SIZE,  # your code
```

```
    shuffle=True,     # your code
    num_workers=4
)
```

```
net_aug = SimpleNN(use_bn=True, activation='relu').to(device)
history_aug, best_aug = train_model(net_aug, train_loader_aug, val_loader,
device,
                                    checkpoint_name="simplenn_aug")
```

Without data augmentation, the best val acc was 68.8%. With augmentation (random crop + horizontal flip), the best val acc improved to 72.96%.

Training acc without augmentation reached 90.80%, while training acc with augmentation was only about 70.16%. Despite the lower training accuracy, val acc was higher with augmentation which means less overfitting.

(b)

```
# BN, part 1 (no-BN vs BN val acc):
net_no_bn = SimpleNN(use_bn=False, activation='relu').to(device)
history_no_bn, best_no_bn = train_model(net_no_bn, train_loader_aug,
val_loader, device,
                                        checkpoint_name="simplenn_no_bn")
```

BN, part 1 (no-BN vs BN val acc):

- Without BN (use_bn=False, lr=0.01): best val acc 70.82
- With BN (use_bn=True, lr=0.01): Best Validation Accuracy = 72.96%
- Conclusion: Adding BN after each conv layer improves val acc by about 2%. BN stabilizes intermediate activations, speeds up convergence, and acts as a mild regularizer.

```
# BN, part 2 (larger LR):
net_bn_lr01 = SimpleNN(use_bn=True, activation='relu').to(device)
history_bn_lr01, best_bn_lr01 = train_model(net_bn_lr01, train_loader_aug,
val_loader, device,
                                            lr=0.1,
checkpoint_name="simplenn_bn_lr01")

net_no_bn_lr01 = SimpleNN(use_bn=False, activation='relu').to(device)
history_no_bn_lr01, best_no_bn_lr01 = train_model(net_no_bn_lr01,
train_loader_aug, val_loader, device,
                                                  lr=0.1,
checkpoint_name="simplenn_no_bn_lr01")
```

BN, part 2 (larger LR):

Model	LR=0.01	LR=0.1
No BN	70.82%	49.70%
With BN	72.96%	71.32%

- No BN with Large LR (lr=0.1): Best val acc = 49.70%
- With BN with Large LR (lr=0.1): Best val acc = 71.32%
- Conclusion: Without BN, a high lr=0.1 leads to unstable gradient updates and poor convergence. BN normalizes layer helps with more consistent scale across layers and allows the network to train stably and achieve strong accuracy even with a 10x lr.

```
# Swish vs ReLU (BN=True, LR=0.1):
net_swish = SimpleNN(use_bn=True, activation='swish').to(device)
history_swish, best_swish = train_model(net_swish, train_loader_aug,
val_loader,
                                        device, lr=0.1,
checkpoint_name="simplenn_swish")
```

Swish vs ReLU (BN=True, LR=0.1):

Activation	Best Val Acc
ReLU	71.32%
Swish	73.66%

- ReLU Activation (lr=0.1): Best val acc = 71.32%
- Swish Activation (lr=0.1): Best val acc = 73.66%
- Conclusion: Swish outperforms ReLU. It also converges more smoothly and reaches its best epoch later (epoch 24) with less oscillation in val acc compared to the ReLU+BN+LR=0.1 run.

(c)

```
lrs = [1.0, 0.1, 0.05, 0.01, 0.005, 0.001]
results_lr = {}

for lr in lrs:
    net_lr = SimpleNN(use_bn=True, activation='relu').to(device)
    history, best_acc = train_model(net_lr, train_loader_aug, val_loader,
device,
                                    lr=lr,
checkpoint_name=f"simplenn_relu_lr_{lr}")
    results_lr[lr] = {"history": history, "best_val_acc": best_acc}

for lr in lrs:
    print(f"lr={lr}: best val acc = {results_lr[lr]['best_val_acc']:.4f}")
```

4c - i (Learning Rate Sweep)

LR	Best Val Acc
1.0	10.72%
0.1	69.66%
0.05	72.50%
0.01	72.60%
0.005	74.04%
0.001	65.62%

A larger learning rate is not always beneficial. At LR=1.0, training completely diverges, loss stays flat around 2.32 and val accuracy stays near 10% throughout. At LR=0.1-0.01, results are fairly close and reasonably strong (69.7%-72.6%). LR=0.005 gives the best result (74.04%), and at LR=0.001 performance drops (65.62%) because training is too slow to converge within 30 epochs.

This shows an inverted-U relationship: learning rate must be tuned, and there is an intermediate range (here around 0.005-0.01) that balances convergence speed and stability.

```
l2s = [1e-2, 1e-3, 1e-4, 1e-5, 0.0]
results_l2 = {}

for l2 in l2s:
    net_l2 = SimpleNN(use_bn=True, activation='relu').to(device)
    history, best_acc = train_model(net_l2, train_loader_aug, val_loader,
device,
                                    lr=0.01, reg=l2,
checkpoint_name=f"simplenn_relu_l2_{l2}")
    results_l2[l2] = {"history": history, "best_val_acc": best_acc}

for l2 in l2s:
    print(f"L2={l2}: best val acc = {results_l2[l2]['best_val_acc']:.4f}")
```

4c - ii (L2 Regularization Analysis)

L2 (reg)	Best Val Acc
1e-2	64.14%
1e-3	71.56%
1e-4	74.18%
1e-5	73.48%
0.0	73.08%

L2 regularization strength has a clear effect on performance, and again follows an inverted-U pattern. Very strong penalty (1e-2) hurts accuracy (64.14%), training accuracy stays low (59%) too, which suggests the model is underfitting due to over-constrained weights. As the penalty is relaxed (from 1e-3 to 1e-4), accuracy improves, and peaks at 1e-4 (74.18%). Going even weaker (1e-5) or removing regularization entirely (0.0) gives similar, slightly lower results (73.48%, 73.08%) — with no L2 penalty, training accuracy is highest (70.3%) but val acc doesn't improve further, consistent with a small amount of regularization helping generalization while too little or none leaves some accuracy on the table (though the effect here is modest compared to the very strong 1e-2 penalty).

This confirms regularization strength is an important hyperparameter.

Bonus

```
# 1. Disable standard L2 weight decay in the optimizer
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9,
weight_decay=0.0)

# 2. Inject L1 penalty directly into the loss function during training
outputs = model(inputs)
loss = criterion(outputs, targets)

# Compute L1 norm across all model parameters
l1_penalty = sum(torch.sum(torch.abs(p)) for p in model.parameters())
loss = loss + (l1_reg * l1_penalty)

loss.backward()
optimizer.step()
```

I re-wrote the train model function. I disabled PyTorch's default L2 weight_decay and manually added the L1 penalty to the loss function.

- **L2 Regularization:** Creates a smooth, bell-shaped curve. It shrinks weights gradually, allowing tiny weights to remain active.
- **L1 Regularization:** Creates a sharp spike exactly at zero. It acts like a strict filter, forcing less important weights completely to zero.

5 Lab (3): Advanced CNN architectures

(a)

```
class SimpleBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                               stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
```

```
                                stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.stride = stride
        self.in_channels = in_channels
        self.out_channels = out_channels

    def _shortcut(self, x):
        # identity case: shapes already match
        if self.stride == 1 and self.in_channels == self.out_channels:
            return x

        # spatially subsample, then zero-pad channels
        x = x[:, :, ::self.stride, ::self.stride]
        channel_diff = self.out_channels - self.in_channels
        pad_before = channel_diff // 2
        pad_after = channel_diff - pad_before
        x = F.pad(x, (0, 0, 0, 0, pad_before, pad_after))
        return x

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + self._shortcut(x)
        return F.relu(out)
```

```
class ResNet20(nn.Module):
    def __init__(self, n=3, num_classes=10):
        super().__init__()

        # stem: single 3x3 conv, 16 filters
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1,
bias=False)
        self.bn1 = nn.BatchNorm2d(16)

        # stage 1: 16 filters, 32x32, no downsampling
        stage1_blocks = []
        for i in range(n):
            stage1_blocks.append(SimpleBlock(16, 16, stride=1))
        self.stage1 = nn.Sequential(*stage1_blocks)

        # stage 2: 32 filters, 16x16, first block downsamples
        stage2_blocks = []
        for i in range(n):
            if i == 0:
                stage2_blocks.append(SimpleBlock(16, 32, stride=2))
            else:
                stage2_blocks.append(SimpleBlock(32, 32, stride=1))
        self.stage2 = nn.Sequential(*stage2_blocks)

        # stage 3: 64 filters, 8x8, first block downsamples
```

```
        stage3_blocks = []
        for i in range(n):
            if i == 0:
                stage3_blocks.append(SimpleBlock(32, 64, stride=2))
            else:
                stage3_blocks.append(SimpleBlock(64, 64, stride=1))
        self.stage3 = nn.Sequential(*stage3_blocks)

        # head: global average pool + fc
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(64, num_classes)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))

        out = self.stage1(out)
        out = self.stage2(out)
        out = self.stage3(out)

        out = self.avgpool(out)
        out = out.view(out.size(0), -1)
        out = self.fc(out)

        return out
```

```
TRAIN_SET_SIZE = len(train_set_aug)   # should be 45000
iters_per_epoch = -(-TRAIN_SET_SIZE // TRAIN_BATCH_SIZE)  # ceil division

milestone_1 = 32000 // iters_per_epoch
milestone_2 = 48000 // iters_per_epoch
total_epochs = 64000 // iters_per_epoch

print(f"iters/epoch: {iters_per_epoch}")
print(f"LR milestones (epochs): {milestone_1}, {milestone_2}")
print(f"Total epochs: {total_epochs}")
```

```
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import MultiStepLR

def train_model(net, train_loader, val_loader, device,
                epochs=30, lr=0.1, momentum=0.9, reg=1e-4,
                milestones=None, gamma=0.1,
                checkpoint_name="model",
checkpoint_folder="./saved_model"):

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum,
weight_decay=reg)
```

```
    if milestones is None:
        milestones = []
    scheduler = MultiStepLR(optimizer, milestones=milestones, gamma=gamma)

    best_val_acc = 0
    history = {"train_loss": [], "train_acc": [], "val_loss": [],
"val_acc": []}

    print(f"==> Training starts! ({checkpoint_name})")
    print("=" * 50)
    for i in range(epochs):
        net.train()
        print("Epoch %d (lr=%.5f):" % (i, optimizer.param_groups[0]['lr']))
        total_examples, correct_examples, train_loss = 0, 0, 0

        for inputs, targets in train_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = net(inputs)
            loss = criterion(outputs, targets)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            _, predicted = outputs.max(1)
            total_examples += targets.size(0)
            correct_examples += predicted.eq(targets).sum().item()

        avg_loss = train_loss / len(train_loader)
        avg_acc = correct_examples / total_examples
        history["train_loss"].append(avg_loss)
        history["train_acc"].append(avg_acc)
        print("Training loss: %.4f, Training accuracy: %.4f" % (avg_loss,
avg_acc))

        net.eval()
        total_examples, correct_examples, val_loss = 0, 0, 0
        with torch.no_grad():
            for inputs, targets in val_loader:
                inputs, targets = inputs.to(device), targets.to(device)
                outputs = net(inputs)
                loss = criterion(outputs, targets)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                total_examples += targets.size(0)
                correct_examples += predicted.eq(targets).sum().item()

        avg_loss = val_loss / len(val_loader)
        avg_acc = correct_examples / total_examples
        history["val_loss"].append(avg_loss)
        history["val_acc"].append(avg_acc)
```

```
        print("Validation loss: %.4f, Validation accuracy: %.4f" %
(avg_loss, avg_acc))

        if avg_acc > best_val_acc:
            best_val_acc = avg_acc
            if not os.path.exists(checkpoint_folder):
                os.makedirs(checkpoint_folder)
            torch.save({'state_dict': net.state_dict(),
                        'epoch': i,
                        'lr': optimizer.param_groups[0]['lr']},
                       os.path.join(checkpoint_folder, f"
{checkpoint_name}.pth"))
        print('')

        scheduler.step()

    print("=" * 50)
    print(f"==> Optimization finished! Best validation accuracy:
{best_val_acc:.4f}")
    return history, best_val_acc
```

The ResNet-20 model was successfully trained across 181 epochs using a multi-step lr schedule, starting at $\eta = 0.1$ with 10× reductions applied at milestones 90 and 136. During the initial phase, accuracy steadily climbed from 36.37% (training) and 46.80% (validation) at Epoch 0 to approximately 92.48% (training) and 87.84% (validation) by Epoch 89. Dropping the lr to 0.01 at Epoch 90 provided a sharp performance boost, immediately driving val acc past the target threshold to 91.66% and peaking at 92.42% by Epoch 97. Subsequent lr reductions allowed training accuracy to converge near 99.51%, while val acc stabilized consistently around 92.30%–92.42% without significant overfitting, achieving a overall best val acc of 92.42%.

`save_test_predictions.ipynb` has the predictions script